Autonomous systems: Autonomous Car Using ROS

Ahmed Mahfouz*, Farid Kamel*, Hazem Mahdy*, Mohamed Shelkamy*,Omar Amin and Loaie Solyman*
*Mechatronics Department - Faculty of Engineering and Materials Science - German University in Cairo, Egypt

***Abstract*—This is a small project for our senior year in the German University in Cairo. The target of this project is to build an autonomous car from scratch and trying to make it move in a certain track using the concepts of locomotion, navigation, localization, mapping, and control. Starting with the locomotion which is deciding how the robot will move and this was done by design our car as a differential 4 wheeled car. Navigation is achieved by identifying the surrounding and checking the safety of the path using the sensors attached to the robot. Localization is obtaining the location of the car relative to the map that it is placed in. The mapping concept is achieved by locating my target, recognizes the path it should take with different path planning techniques while avoiding any obstacles in its way. Lastly, the control architecture that connects between the hardware and how it should behave using a software code done on Robot Operating System (ROS) and a simulation of the robot was done on Gazebo software. Finally, after finishing the hardware and finishing the connections, the control and all the previous concepts, the robot was able to complete its path successfully and avoid any obstacles on its way to the finish line.**

***Index Terms*—Autonomous vehicle, ROS, control, path planning, Gazebo, robots. , multi-robot systems, ROS, Localization**

I. Introduction

The idea of autonomous, multi-robot systems (**MRS**), and the synchronization of distributed agents or controllers have been attained in the last few decades. The reason behind the evolution in this field is the outstanding advantages of (**MRS**) compared to single-robot systems [1].

Multi-robot systems can go with tasks with the consideration of the five challenges of the autonomous systems, which are Locomotion, Navigation, Localization, Mapping, and Control architecture. Nowadays, the application of the algorithms used to handle these challenges can be implemented and simulated using the Robot Operating System (**ROS**), which is one of the most known and practical frameworks for robot software development. It is used widely in the fields of industrial manipulators and mobile or service robots in a wide range of task complexity [2].

This paper will concentrate on the analysis of the model of a mobile robot with the category (deferential or Ackerman) robot which (utilize two independent wheels for actuation and the 3rd wheel for stability) or (which uses the rear one-shaft wheels for controlling the robot speed and the front two speeds for the steering).

Due to scalability, robustness, reliability, and cost considerations, multi-robot systems are favored over single robot systems in complex missions such as unknown and unstructured area exploration. However, several practical issues must be resolved before such systems can be realized. We demonstrate a generic, low-power, compute-efficient hardware framework in this paper.

II. State of Art

The validity and efficiency of the AVERT robotic system are illustrated via experiments in an indoor parking lot. Demonstrating successful autonomous navigation, docking, lifting, and transportation. The overall developed system applies to reason about available trajectory paths, wheel identification, local and undercarriage obstacle detection. The novel lifting robots are capable of omnidirectional movement. Thus they can under-ride the desired vehicle and dock to its wheels for a synchronized lifting, and extraction [3].

Paper [4] illustrates through Simulations that not only the accuracy of localization, and mobility tracking could be greatly enhanced in general, but also the robustness can be guaranteed under circumstances where traditional localization and tracking devices fail. Moreover, a multi-sensor multi-vehicle localization and mobility tracking framework is developed for autonomous vehicles equipped with GPS, an inertial measurement unit (IMU), and an integrated sensing system.

Multi-vehicle deployment can be formulated as a cooperation formation problem. The system of weighted consensus is used. The authors' objective was to make the adjacent vehicles with constant space between them while keeping a common velocity and acceleration, also the platoon of these vehicles has to converge fast and has strong robustness when there is a disturbance in the system, so to achieve this a third-order dynamics are established for vehicle platoon that is regarded as connected vehicle systems (CVSs) and distributed control protocols are designed to implement cooperation control [5].

In [6], the main problem is a path planning optimization problem for a group of vehicles together to perform certain deliveries. This group consists of a truck that can only move in the streets and a smaller vehicle called quadrotor micro-aerial vehicle of capacity one that can be launched from the truck to perform deliveries. The target is formulating the shortest cooperative route allowing the quad-rotor to deliver items at all requested locations. Two algorithms were used based on enumeration and a reduction to the traveling salesman problem to indicate the results of this problem. Simulation results compare the performance of the two algorithms and explain delivery times on real-life roads.

The authors presented how a team of robots can complete complex tasks using a common, low-power, compute-efficient hardware platform. Using a team of four robots, this platform is used to demonstrate an exploration task in an unknown area. The overall review and findings for the demo application

are also addressed, as well as the functional difficulties of implementing such multi-agent systems, and they conclude that Building robust robotic systems to operate in unknown environments is a challenging frontier in robotics research. Such systems must operate for extended periods in complex domains to accomplish a given mission. This requires various system-level issues to be addressed [1].

Paper [7] discusses the multi-vehicle localization parameters that make up a robust and precise vehicle basing this centimeter-level localization on sensor fusion using GNSS, LiDAR, and IMU. An error-state Kalman filter is applied to fuse the localization measurements, thus a whole system architecture was developed localizing the readings from LiDAR and the GNSS allowing for very large amounts of data to be driven from 60 Km span in diverse city scenes.

In [8], the authors resolve the problem of time delays in vision-based multi-robot systems. These delays can be different and complex due to image processing, communication network, and camera latency. The main target is focusing on fixed-time formation control of multi-robot systems that are subjected to delay problems. A predictor-based state transformation is inserted to deal with the input delay for each robot. Then, nonlinear fixed time formation protocols are placed for the robots and the corresponding settling time is obtained by Lyapunov functions. Finally, the protocols are run through a numerical simulation model and then implemented on an E-puck robots platform.Both simulation and experimental results showed the effectiveness of the formation protocols .

In paper [9], the goal is to have the robots coordinate, with each other to achieve a set of tasks to maximize the search area and minimize the overall exploration time. For the robots, to learn cooperatively, a multi-robot cooperative learning approach for a hierarchical reinforcement learning (HRL), based semiautonomous control architecture was developed. The proposed approach allows effective task allocation among the multi-robot team and efficient execution of the allocated tasks .

In paper [10], the control and monitoring of temperature and humidity are done by using mobile sensory systems. The paper addresses some collateral challenges of multi-robot systems, such as mission planning or guidance, navigation, and control. The mobile robot teams include ground and aerial vehicles to provide flexibility and improve performance .

The authors' main focus in [11] was the fundamental element in the autonomous driving mindset that is the sensing element, tackling the cooperative sensing which allows for cost reduction, high accuracy and overcoming vision range limit. By characterizing the uncertainty of the vehicle detection results and designing four probability distribution algorithms increasing the overall sensing range and sensing accuracy of the multi vehicle grid contractions.

III. Methodology

A. Hardware

As shown in figure 1, the design for our robot consists of two parallel plates on which 4 DC motors are mounted.

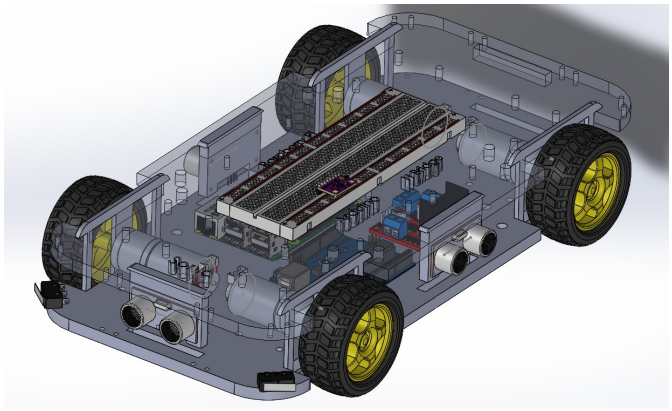

Fig. 1: Solidwork design for the robot chassis

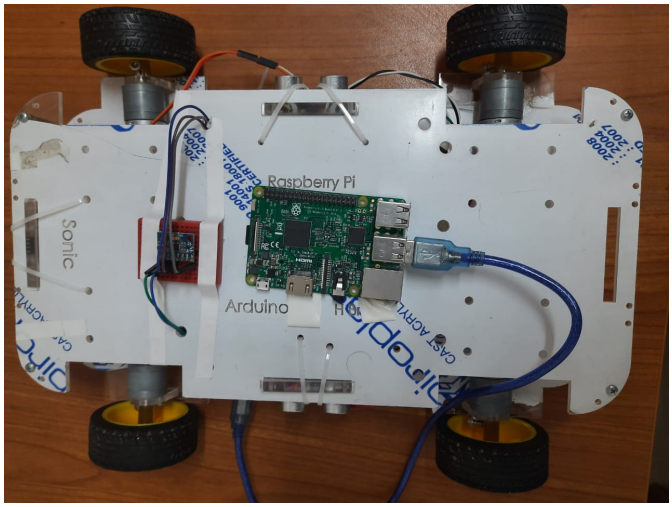

Fig. 2: Assembly of the manufactured hardware

As a differential robot, each two wheels will be controlled via one control signal, for example, the two DC motors of the right wheels will be controlled by one control signal, so they perform the same motion.

The DC motors, the motor drivers, the Arduino controller and the Raspberry pi will be fixed on the lower plate. The four robot wheels tires will be mounted on the motors' shafts.The IMU sensor and the bread board will be fixed on the upper plate for angle and steering control . Three ultrasonic sensors will be fixed vertically between the two plates, in addition two limit switch sensors for obstacle avoiding tasks. Figure 2 shows the assembly of the manufactured robot hardware.

B. Modelling

The states used in the kinematic modelling of the differential robot are $[x, y, \theta]$ where x is the position in the x-axis, y is the position in the y-axis and θ is the orientation of the robot. The velocity of the wheel is defined as $V = \omega r$ where ω is the angular velocity of the wheel, r is the radius of the wheel

and V is the transnational velocity.

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \frac{R}{2}\cos\theta \\ \frac{R}{2}\sin\theta \\ \frac{R}{L} \end{pmatrix} \dot{\phi_R} + \begin{pmatrix} \frac{R}{2}\cos\theta \\ \frac{R}{2}\sin\theta \\ -\frac{R}{L} \end{pmatrix} \dot{\phi_L} \quad (1)$$

C. Control

The husky vehicle is considered an under actuated system. Under actuated systems are those systems where the number of control inputs is less than the number of the degrees of freedom. Controlling such systems is a difficult task because there are not enough inputs to control all the degrees of freedom. Two different controllers have been used on the husky robot to overcome this problem. The results of the two controller, Point-to-Point Controller and Lyapunov based controller have been presented and compared.

1) Point-to-Point (P2P) controller: It is a state feedback type controller. A transformation has been made from the coordinates of the system x, y, and θ to polar coordinates ρ, β, α to facilitate the construction of the controller. The control law used for the linear velocity:

$$v = k_\rho \rho \quad (2)$$

And for the angular velocity:

$$\omega = k_\alpha \alpha + k_\beta \beta \quad (3)$$

2) Lyapunov Controller: It is an energy based approach. A selected function is used to represent the energy of the system and its derivative represents the decaying of that energy till it reaches zero at the desired position. The control law for the linear velocity:

$$v = v_d cos(\theta) + k_x x_r \quad (4)$$

And for the angular velocity:

$$\omega = v_{dyr} + w_d + k_\theta sin(\theta_r) \quad (5)$$

IV. Gazebo Platform

Gazebo platform has been used to simulate different ground robots' models such as Ackerman Mechanism, differential robot, and turtle boat. Gazebo has the advantage of making a complex environment to simulate all different types of robots both indoor and outdoor. It is also capable of simulating very complex motions of different types of robot easily and efficiently. Gazebo is user friendly and easy to understand, use and modify.

Gazebo platform has been utilized to show the performance of the robot and to acquire the data in the Results section. It also has been used to simulate a complex environment for the differential robot as shown in the following figure:

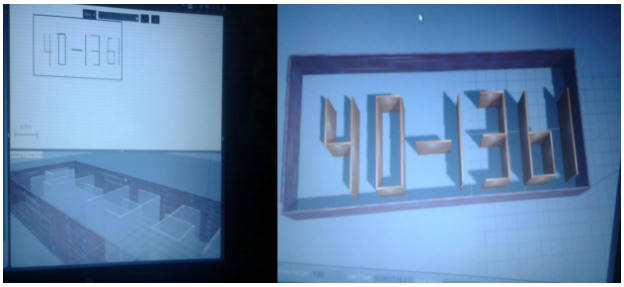

Fig. 3: Map Design using Build Editor

V. Results

A. Test Cases and Parameters

The following table represents different cases for the controllers. The table shows the initial conditions as well as the desired value for x, y and θ.

Case Num.	x_0	y_0	θ_o	x_{desire}	y_{desire}	θ_{desire}
1	2	0	0	10	10	90
2	2	0	180	-3	5	90
3	2	-3	0	1	3	180
4	6	-8	0	-6	8	0
5	-6	9	0	4	3	180
6	-3	5	90	-6	8	90
7	-5	-3	180	5	3	90
8	-3	-5	180	-6	8	180

TABLE I: Test cases

The results shown in figures4:9 that both of the controllers were able to reach the desired values, but with steady state errors. Lyapunov controller showed a more stable behavior than the point to point controller which was not robust enough in some positions and causes oscillations in the error values and the controller effort.

Furthermore, Lyapunov could have better performance if there was a desired trajectory instead of a fixed desired position as this would help to get the inverse kinematics of the robot which results in getting the desired linear and angular velocity required to reach the desired position with much less error than the one happened during the simulation.

Table IIIshows the values of the parameters used in both controllers

k_ρ	k_α	k_β
0.5	1.5	-0.6

TABLE II: Parameters value for P2P controller

k_x	k_θ	v_d	ω_d
2.5	0.5	1.0	0.5

TABLE III: Parameters value for P2P controller

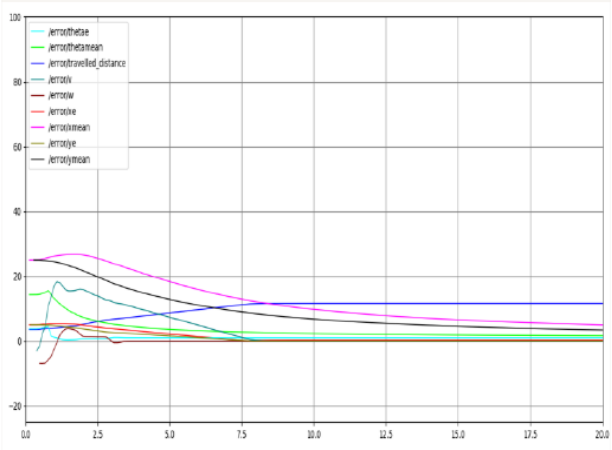

Fig. 4: Test case 2 Lyapunov graph

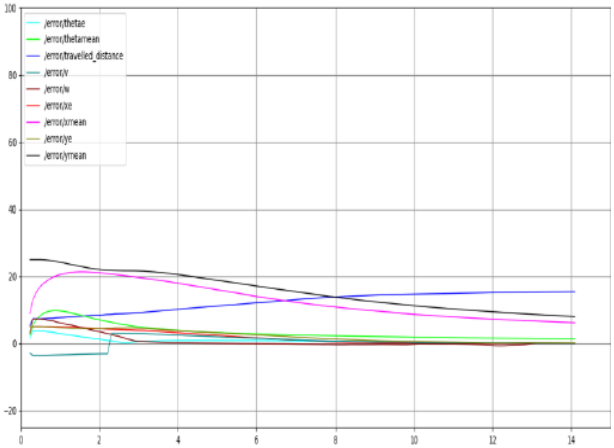

Fig. 5: Test case 2 point to point graph

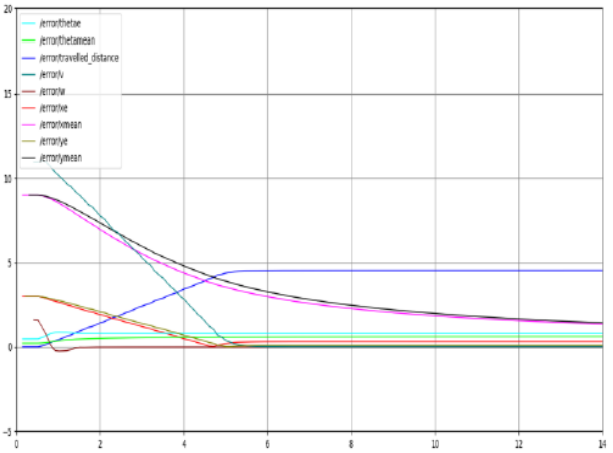

Fig. 6: Test case 4 Lyapunov graph

B. Open loop Test

In this section, the robot was given a set of desired positions to go to from different initial positions. The robot was observed that it tries to reach the desired position but there is still a small error in the position of the robot with respect to the desired one. The robot node will communicate through topics to send and receive messages. For the robot node to receive a message, publications are used with appropriate message for controlling of the husky robot. As well as subscriptions with appropriate messages for assessing the robot's position which will then be used for the open loop response of the robot under a pre-listed code for the control, and for the teleportation which will utilize keyboard keys for the up/down/right/left motion of the husky robot. All of the actions listed required the messages to be published to and subscribed for the robot in order to create simple read/write operations on the robot's node.

These trajectories of straight line and circular path were also implemented unto the hardware created for the project using the high level controller (raspberry pi) publishing to the low level controller (Arduino) which in turn sends actuation potential difference across the dc motors to allow for actuation in a specific manner that performs the desired trajectories.

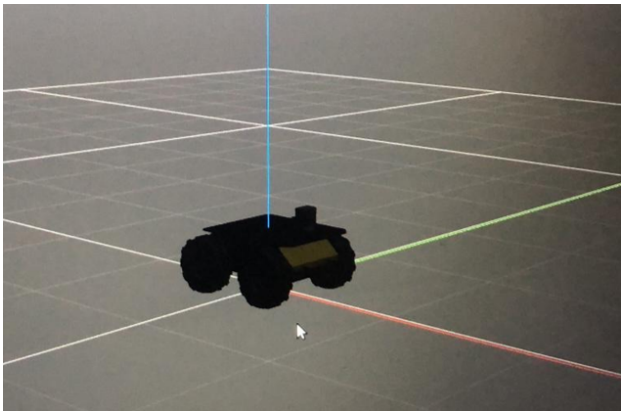

Fig. 7: The Husky Robot model in Gazebo world performing the trajectory

C. Serial communication Test

1) LED tests: One of the experiments that was to test the communication protocols between the Arduino (low level controller) and the Raspberry Pi (high level controller). The code started by creating ROS nodes that publish and subscribe messages to and from the Arduino. A launch file was created to run these nodes and there were two tests done using 2 different implementations that were run from the launch file.

The first test was writing a message on the high level controller and sending it to be read by the low level controller. The output of the received message was shown on an LED connected to the Arduino pins.

The second experiment was similar to the first one but instead the node of the high level controller should receive a message coming from the low level controller, the output was also projected on an LED but this time the LED is connected to the Pi's pins. Both tests were a success and the LED indicated the received messages.

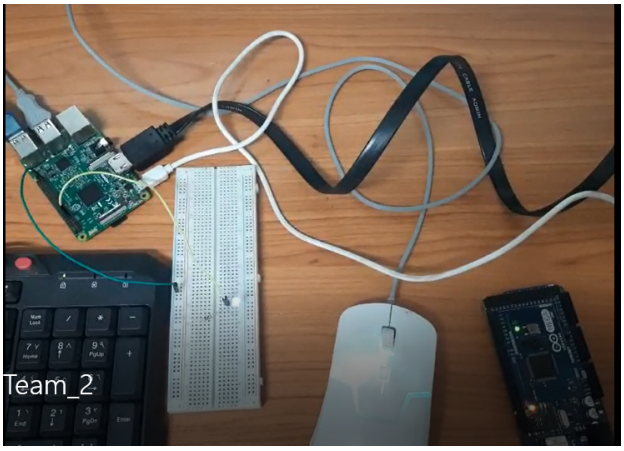

Fig. 8: The LED Experiment Implementation

2) Sensors Experiment: This experiment was implemented based on the LED experiment, where the data of the sensors were collected using the low level controller (Arduino) and published to the high level controller (Raspberry-pi) using ROS nodes. This experiment resulted in a successful communication between the Arduino and the Raspberry-Pi. Arduino receives the desired position and speed from the Raspberry-Pi which also receives the data of the Ultrasonic and IMU sensors from the Arduino.

D. Closed loop Test

Here, the data was acquired by the sensors about the current position of the robot and then it was send as a feedback to the controller to identify whether the robot is close to the desired position or not. So that, the robot can handle going to a desired position or following a specific trajectory even if external disturbances occurred during the motion. As shown in the controller section, The controller manage to stabilize the robot at the desired positions 5 and 9 by using point to point controller and 4 and 8 by using Lyapunov based controller. Here, another test case is going to be presented for both controllers.

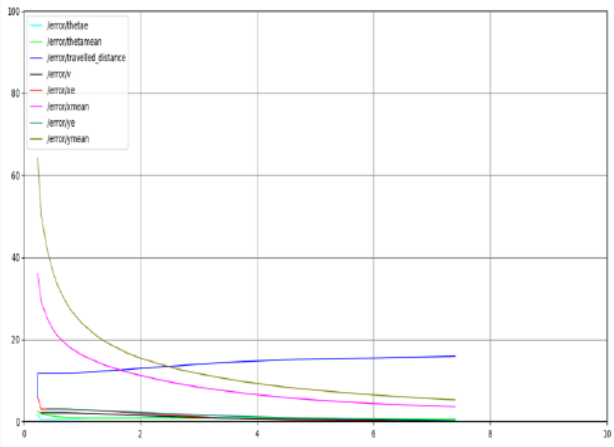

Fig. 9: Test case 4 point to point graph

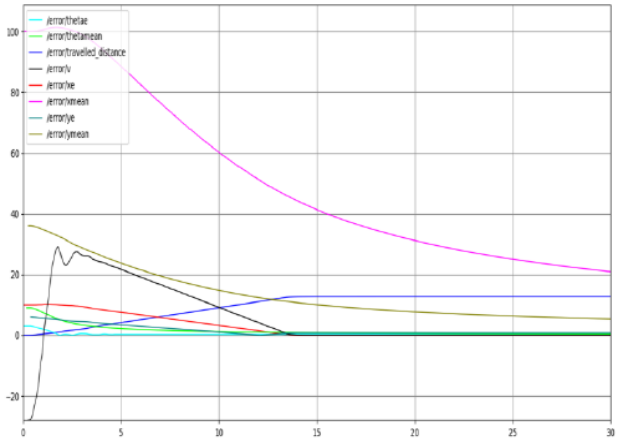

Fig. 10: Test case 6 Lyapunov graph

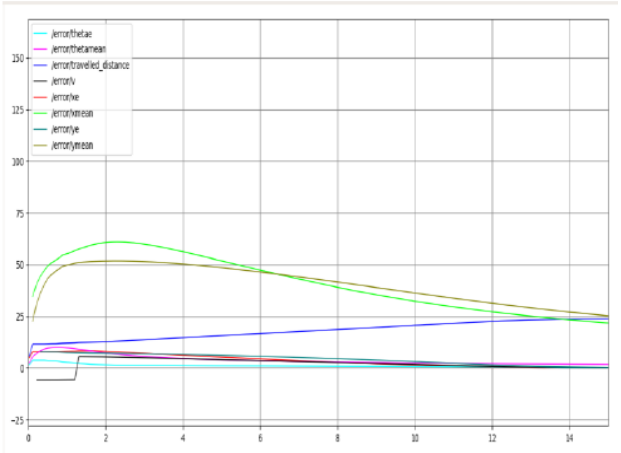

Fig. 11: Test case 6 point to point graph

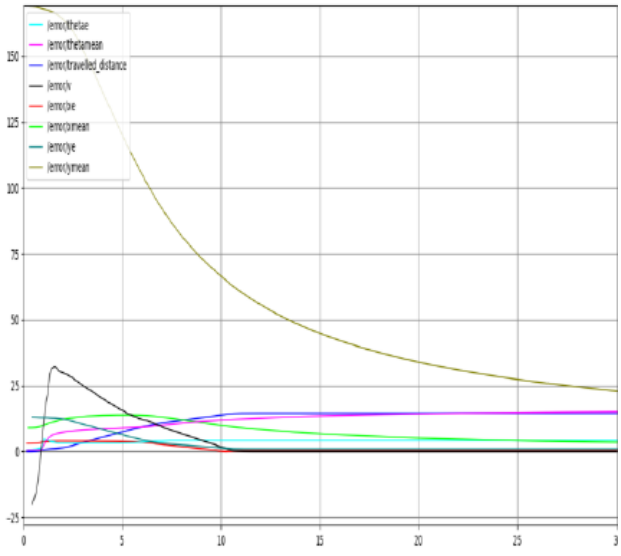

Fig. 12: Test case 8 Lyapunov graph

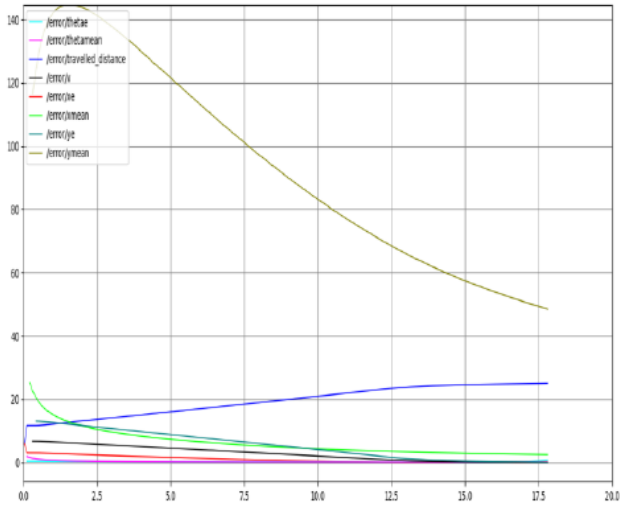

Fig. 13: Test case 8 point to point graph

As shown from the previous figures, Lyapunov based controller developed a better response than the point to point.

The ROS nodes created were Trajectory Planning Node which subscribes the filtered initial position of the robot published by the filtration Node. The Control Node which subscribes the filtered position of the robot published by the filtration node. The Communication Node which establishes the protocol between high and low level control boards to acquire sensory data and actuate motor angular velocities. The Filtration Node which takes the noisy data given by the localization sensors (Encoders, IMU, ... etc.) and after implementing the Kalman Filter, you can estimate the correct position of the robot (Xrob, Yrob, Thetarob).

The closed loop response of the mobile vehicle was tested in a track shown below that utilizes the straight line trajectory and a subset of the circular path. All nodes created were set to perform each task required from them in order to achieve smooth autonomous motion of the vehicle in the specified path.

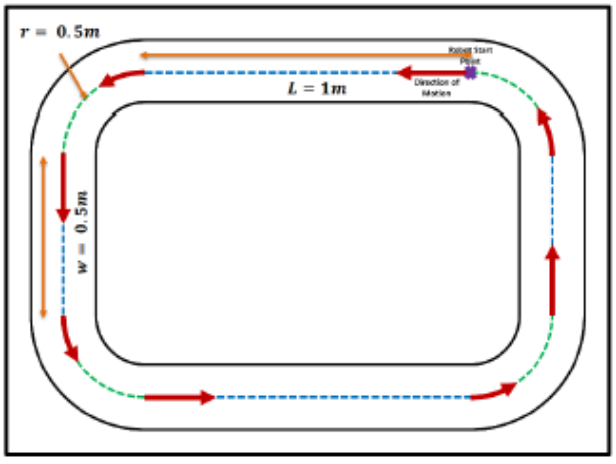

Fig. 14: Closed Loop Track Dimensions

VI. Kalman filter

IN the system, the kalman filter , which is a recursive algorithm used to generate optimal estimate of the states of a system from a series of incomplete and noisy measurements, is used in order to get the optimal path for the robot. The Kalman Filter has certain steps in order to operate which are:

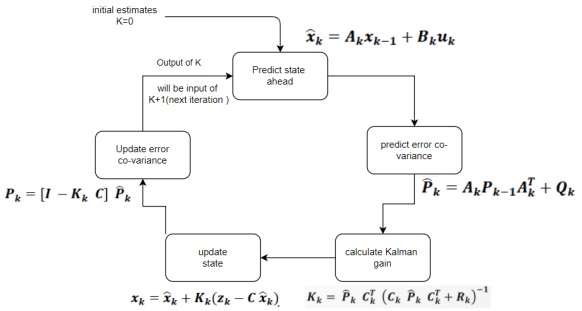

Fig. 15: Kalman filter sequence and equations

In this project, a sensor filtration node is designed to implement a Kalman Filter in Raspberry Pi. This node takes the data with noise measured by the localization sensors as the Encoders, IMU and ultrasonic and after implementing the filter, the correct position of the robot $x_{rob}, y_{rob}, \theta_{rob}]$ can be estimated.

VII. Conclusion and Future Recommendations

As a conclusion, these promising results were deployed onto the hardware that was fully prepared to start receiving sensory data and accept actuation orders from the low level controller (Arduino) that was calculated using P2P or Lyaponauv control laws in the High level controller (Raspberry pi) and the first few iterations of the robot to show autonomy were the OLR of trajectories such as straight line and circular paths at different percentages of maximum possible velocity of the robot. Then, Closed loop response was performed for the same trajectories showing an actual autonomous system of a mobile robot that can be expanded unto many different applications of the autonomous industry, tackling obstacles and changing trajectories while in iteration by defining new goal positions. These results can be expanded to embed intersection management, autopilot operations of auto-parking, harness back to key-control and other applications where autonomy is required.

References

[1] V. Honkote, D. Ghosh, K. Narayanan, A. Gupta, and A. Srinivasan, "Design and integration of a distributed, autonomous and collaborative multi-robot system for exploration in unknown environments," in *2020 IEEE/SICE International Symposium on System Integration (SII)*. IEEE, 2015, pp. 1232–1237.

[2] J. J. Roldán, E. Peña-Tapia, D. Garzón-Ramos, J. de León, M. Garzón, J. del Cerro, and A. Barrientos, "Multi-robot systems, virtual reality and ros: developing a new generation of operator interfaces," in *Robot Operating System (ROS)*. Springer, 2019, pp. 29–64.

[3] A. Amanatiadis, C. Henschel, B. Birkicht, B. Andel, K. Charalampous, I. Kostavelis, R. May, and A. Gasteratos, "Avert: An autonomous multi-robot system for vehicle extraction and transportation," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 1662–1669.

[4] P. Yang, D. Duan, C. Chen, X. Cheng, and L. Yang, "Multi-sensor multi-vehicle (msmv) localization and mobility tracking for autonomous driving," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 14 355–14 364, 2020.

[5] Y. Liu, C. Zhai, and H. Gao, "Autonomous multi-vehicle consensus for high-order systems," in *2017 36th Chinese Control Conference (CCC)*. IEEE, 2017, pp. 9548–9553.

[6] S. L. W. Neil Mathew, Stephen L. Smith, "Planning paths for package delivery in heterogeneous multirobot teams," in *IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING (SII)*. IEEE, 2015, pp. 1298–1308.

[7] R. C. H. L. Y. Z. H. W. S. S. Guowei Wan, Xiaolong Yang, "Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[8] Z. Z. B. L. Z. D. Chunyan Wang, Hilton Tnunay, "Fixed-time formation control of multi-robot systems: Design and experiments," in *IEEE Transcations on industrial electronics*. IEEE, 2018.

[9] Y. Liu and G. Nejat, "Multirobot cooperative learning for semiautonomous control in urban search and rescue applications," *Journal of Field Robotics*, vol. 33, no. 4, pp. 512–536, 2016.

[10] J. J. Roldán, P. Garcia-Aunon, M. Garzón, J. De León, J. Del Cerro, and A. Barrientos, "Heterogeneous multi-robot system for mapping environmental variables of greenhouses," *Sensors*, vol. 16, no. 7, p. 1018, 2016.

[11] C. C. X. C. Yuru Li, Dongliang Duan and L. Yang, "Occupancy grid map formation and fusion in cooperative autonomous vehicle sensing," in *IEEE International Conference on Communication Systems (ICCS)*, 2018.